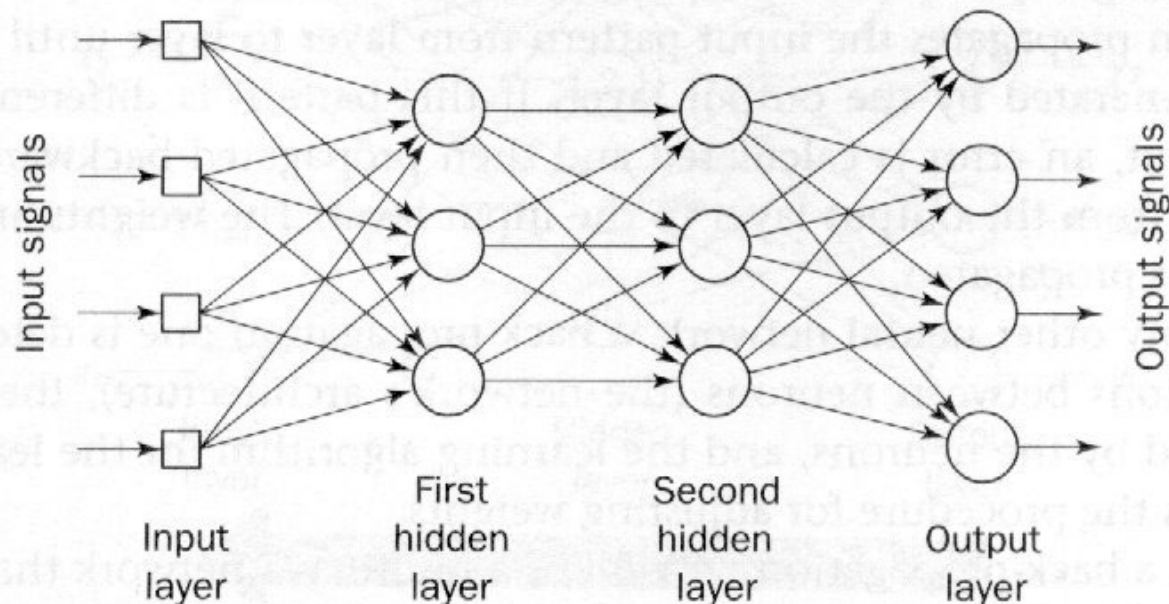


Class 04A : Multilayer Neural Networks



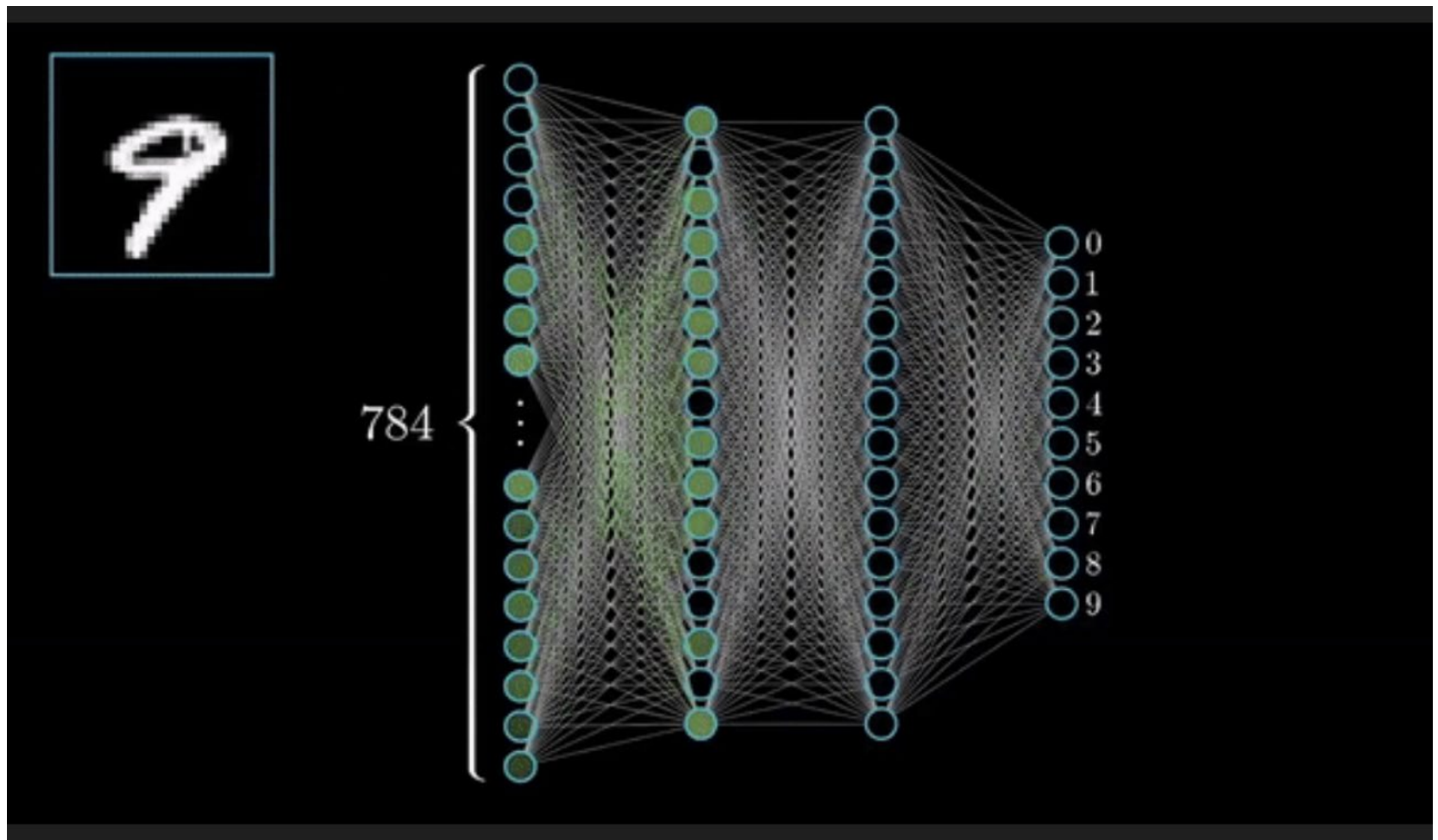
What is a multilayer neural network ?

- A multilayer neural network is a **feedforward** neural network with one or more **hidden layers**.
- The network consists of an input layer of source neurons, at least one middle or hidden layer of computational neurons, and an output layer of computational neurons.

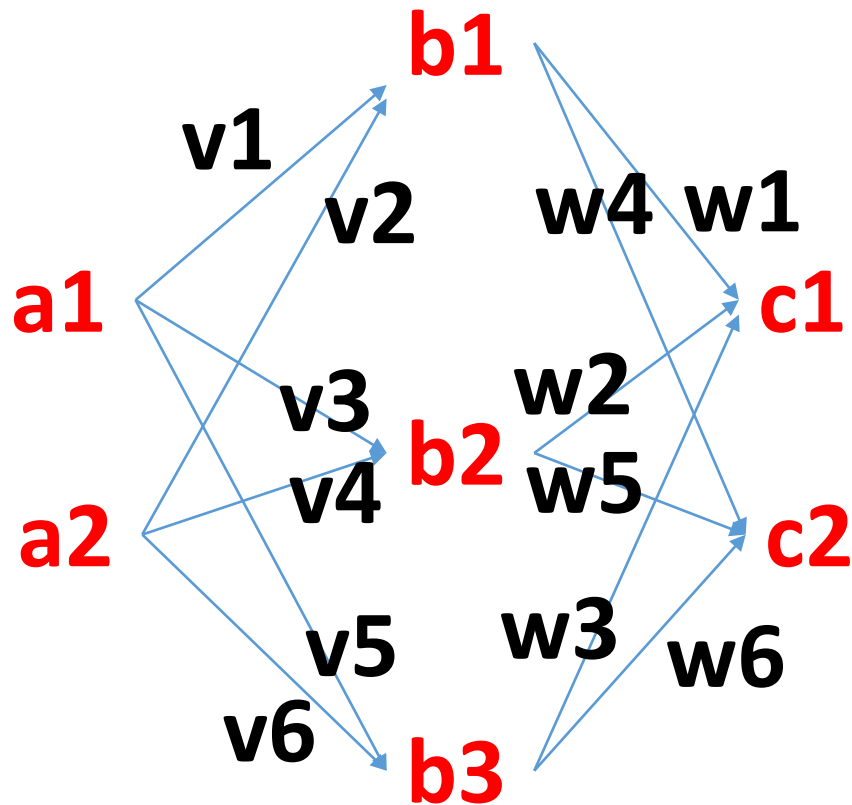


Why do we need a hidden layer ?

- Neurons in the hidden layer detect the features; the weights of the neurons represent the features hidden in the input patterns.
- These features are then used by the output layer in determining the output pattern.
- With one hidden layer, we can represent any continuous function of the signals. With two hidden layers, even discontinuous functions can be represented.



MNIST: ANN 1 or more **10 outputs**
784 inputs hidden layers
(28x28) with hidden
 units



Basic equations:

$$\begin{bmatrix} b1 \\ b2 \\ b3 \end{bmatrix} = \begin{bmatrix} v1 & v2 \\ v3 & v4 \\ v5 & v6 \end{bmatrix} \begin{bmatrix} a1 \\ a2 \end{bmatrix}$$

$$\begin{bmatrix} c1 \\ c2 \end{bmatrix} = \begin{bmatrix} w1 & w2 & w3 \\ w4 & w5 & w6 \end{bmatrix} \begin{bmatrix} b1 \\ b2 \\ b3 \end{bmatrix}$$

ANN Feedforward equations
(f sigmoid):

$$\begin{bmatrix} b1 \\ b2 \\ b3 \end{bmatrix} = f\left(\begin{bmatrix} v1 & v2 \\ v3 & v4 \\ v5 & v6 \end{bmatrix} \begin{bmatrix} a1 \\ a2 \end{bmatrix} + \begin{bmatrix} \theta1 \\ \theta2 \\ \theta3 \end{bmatrix}\right)$$

$$\begin{bmatrix} c1 \\ c2 \end{bmatrix} = \begin{bmatrix} w1 & w2 & w3 \\ w4 & w5 & w6 \end{bmatrix} \begin{bmatrix} b1 \\ b2 \\ b3 \end{bmatrix} + \begin{bmatrix} \tau1 \\ \tau2 \end{bmatrix}$$

Matrix-Vector multiplication

Feed Forward Propagation:

2 inputs, 3 hidden units, 3 outputs

How many parameters ?

Learning Rule

- The powerful learning rule used for updating (training) the network weights is called the **backpropagation** generalized delta rule (GDR).
- It is a form of **gradient descent** applicable to units whose net activation, net_i , is a **sigmoid** function (or purelin or tanh).
- The **training instances** set for training the network must be presented many times in order for the interconnection weights between the neurons to settle into a state for correct classification of input patterns.
- **The GDR is a learning rule for a feedforward, multiple-layer, structured neural network that uses gradient descent to achieve training or learning by error correction.**
- **Network weights are adjusted to minimize an error based on a measure of the difference between desired and actual feedforward network output.** Desired input/output behavior is given in the training set.

The Generalized Delta Rule (GDR)

The process consists of the following steps:

1. **Initialize** all unit weights in the network.
2. **Apply an input** (stimulus) vector to the network.
3. **Feed forward** or propagate the input vector to determine all unit outputs.
4. **Compare** unit responses in the output layer with the desired or target response.
5. **Compute** and propagate an error sensitivity measure backward (starting at the output layer) through the network, using this as the basis for weight correction.
6. Minimize the overall error at each stage through unit **weight adjustments**.

The following notation is used:

a: input pattern (vector)

c: corresponding output pattern or response (vector)

c^k: desired (or target) system output (vector).

The basis of the algorithms to be derived is gradient descent.

To adjust network weights, we define and compute a mapping error E and the gradient $\frac{\partial E}{\partial w_{ij}}$.

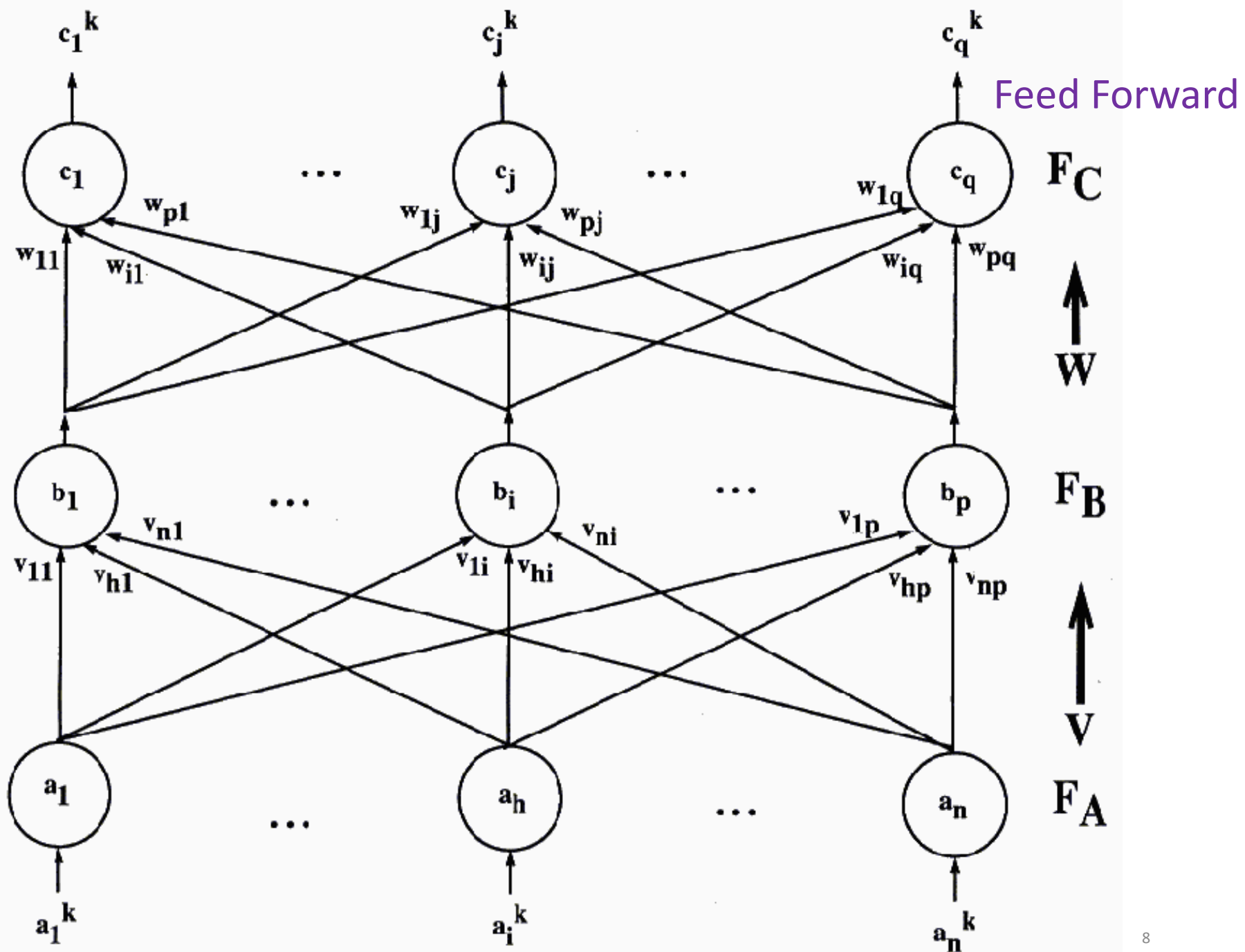
The weight w_{ij} adjustment is then set proportional to $-\frac{\partial E}{\partial w_{ij}}$.

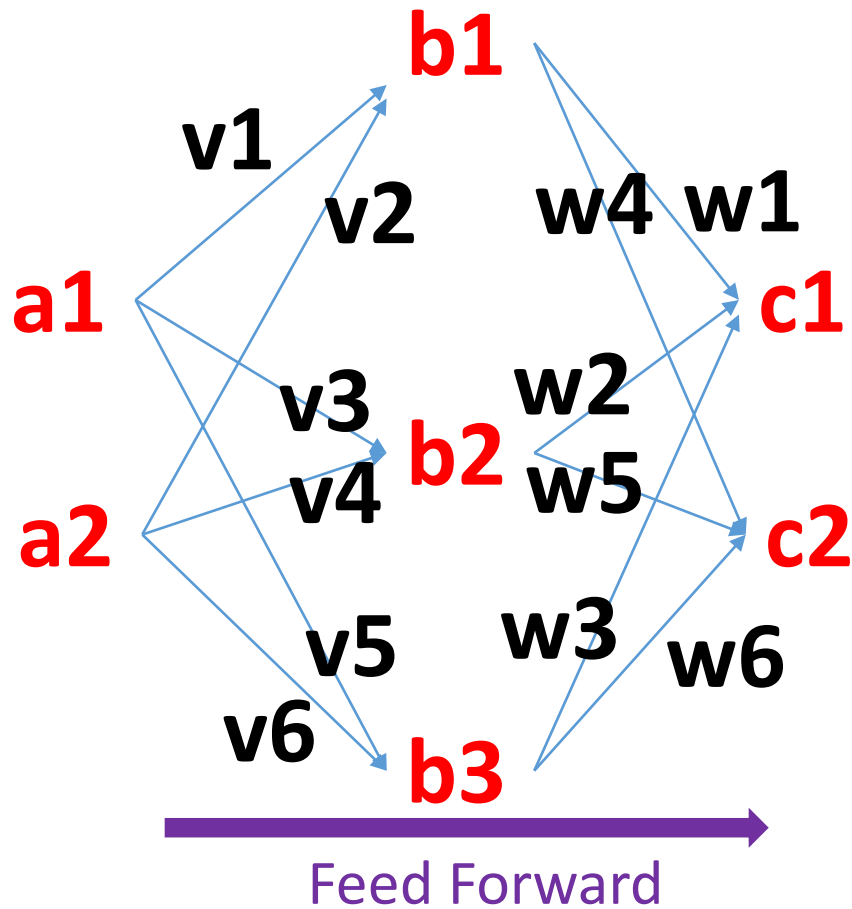
The elementary backpropagation is a three-layer ANN with feedforward connections from the F_A processing elements (PEs) to the F_B PEs and feedforward connections from the F_B PEs to the F_C PEs.

The error which we want to minimize is: $E = \frac{1}{2} \sum_{j=1}^q (c_j^k - c_j)^2$

E is squared error. (Mean Squared Error is MSE)

The figure is shown below.





Basic equations:

$$\begin{bmatrix} b1 \\ b2 \\ b3 \end{bmatrix} = \begin{bmatrix} v1 & v2 \\ v3 & v4 \\ v5 & v6 \end{bmatrix} \begin{bmatrix} a1 \\ a2 \end{bmatrix}$$

$$\begin{bmatrix} c1 \\ c2 \end{bmatrix} = \begin{bmatrix} w1 & w2 & w3 \\ w4 & w5 & w6 \end{bmatrix} \begin{bmatrix} b1 \\ b2 \\ b3 \end{bmatrix}$$

ANN forward equations (f sigmoid):

$$\begin{bmatrix} b1 \\ b2 \\ b3 \end{bmatrix} = f \left(\begin{bmatrix} v1 & v2 \\ v3 & v4 \\ v5 & v6 \end{bmatrix} \begin{bmatrix} a1 \\ a2 \end{bmatrix} + \begin{bmatrix} \theta1 \\ \theta2 \\ \theta3 \end{bmatrix} \right)$$

$$\begin{bmatrix} c1 \\ c2 \end{bmatrix} = \begin{bmatrix} w1 & w2 & w3 \\ w4 & w5 & w6 \end{bmatrix} \begin{bmatrix} b1 \\ b2 \\ b3 \end{bmatrix} + \begin{bmatrix} \tau1 \\ \tau2 \end{bmatrix}$$

Matrix-Vector multiplication

Feed Forward Propagation:

2 inputs, 3 hidden units, 3 outputs

Backpropagation (BP) Algorithm

for sigmoid hidden units and purelin output units

1. Assign random values in the range $[-1, +1]$ to all the F_A to F_B inter-layer connections, v_{hi} , all the F_B to F_C inter-layer connections, w_{ij} , to each F_B PE threshold, θ_i , and to each F_C PE threshold, Γ_j .
2. For each pattern pair (A_k, C_k) , $k = 1, 2, \dots, m$, do the following:
 - (a) Transfer A_k 's values to the F_A PEs, filter the F_A PE activations through V and calculate the new F_B PE values (activations) using the equation

$$b_i = f\left(\left[\sum_{h=1}^n a_h v_{hi}\right] + \theta_i\right)$$

for all $i = 1, 2, \dots, p$, where b_i is the activation value of the i th F_B PE, θ_i is the i th F_B PE's threshold value, and f is the logistic sigmoid threshold function :

$$f(x) = \frac{1}{1 + e^{-x}}$$

- (b) Filter the F_B activations through W to F_C using the (*purelin*) equation

$$c_j = \left(\sum_{i=1}^p b_i w_{ij} \right) + \Gamma_j$$

for all $j = 1, 2, \dots, q$, where c_j is the activation value of the j th F_C PE, Γ_j is the j th F_C PE's threshold value.

- (c) Compute the discrepancy (error) between the computed and desired F_C PE values using the equation

$$d_j = c_j^k - c_j$$

for all $j = 1, 2, \dots, q$, where d_j is the j th F_C PE's computed error.

- (d) Calculate the error of each F_B PE relative to each d_j with the equation for all $i = 1, 2, \dots, p$, where e_i is the i th F_B PE's computed error.

$$e_i = b_i(1 - b_i) \sum_{j=1}^q w_{ij} d_j$$

- (e) Adjust the F_B to F_C connections for all $i = 1, 2, \dots, p$, where $j=1, 2, \dots, q$. α is a positive constant controlling the learning rate.

$$w_{ij}^{new} = w_{ij}^{old} + \Delta w_{ij}$$

$$\Delta w_{ij} = \alpha b_i d_j$$

- (f) Adjust the F_C thresholds for all $j = 1, 2, \dots, q$, where $\Delta \Gamma_j$ is the amount of change to the j th F_C PE's threshold value.

$$\Gamma_j^{new} = \Gamma_j^{old} + \Delta \Gamma_j \quad \text{where} \quad \Delta \Gamma_j = \alpha d_j$$

(g) Adjust the F_A to F_B connections for all $h = 1, 2, \dots, n$, and all $i = 1, 2, \dots, p$.

β is a positive constant controlling the learning rate.

$$v_{hi}^{new} = v_{hi}^{old} + \Delta v_{hi} \quad \text{where } \Delta v_{hi} = \beta a_h e_i$$

(h) Adjust the F_B thresholds by

$$\theta_i^{new} = \theta_i^{old} + \Delta \theta_i \quad \text{where } \Delta \theta_i = \beta e_i$$

for all $i = 1, 2, \dots, n$, where $\Delta \theta_i$ is the amount of change to the i^{th} F_C PE's threshold value.

3. Repeat step (2) until the error correction value d_j , for each $j = 1, 2, \dots, q$, and each $k = 1, 2, \dots, m$, is either sufficiently low or zero.

4. One forward run of the ANN (also called Backpropagation recall) accepts an input at the F_A PEs and feeds it through the V and W matrices to produce a response at the F_C PEs. This feedforward operation creates F_B PEs (activations) using

$$b_i = f\left(\left(\sum_{h=1}^n a_h v_{hi}\right) + \theta_i\right)$$

for $i = 1, 2, \dots, p$. Once the F_B PE values have been all calculated they are then used to create new F_C PE values using

$$c_j = \left(\sum_{i=1}^p b_i w_{ij}\right) + \Gamma_j$$

for all $j = 1, 2, \dots, q$.

Backpropagation (BP) Algorithm

for sigmoid hidden units and sigmoid output units

1. Assign random values in the range $[+1, -1]$ to all the F_A to F_B inter-layer connections, v_{hi} , all the F_B to F_C inter-layer connections, w_{ij} , to each F_B PE threshold, θ_i , and to each F_C PE threshold, Γ_j .
2. **Scale the inputs and outputs.** For each pattern pair (A_k, C_k) , $k = 1, 2, \dots, m$, do the following:
 - (a) Transfer A_k 's values to the F_A PEs, filter the F_A PE activations through V and calculate the new F_B PE values (activations) using the equation

$$b_i = f\left(\left(\sum_{h=1}^n a_h v_{hi}\right) + \theta_i\right)$$

for all $i = 1, 2, \dots, p$, where b_i is the activation value of the i th F_B PE, θ_i is the i th F_B PE's threshold value, and f is the logistic sigmoid threshold function :

$$f(x) = \frac{1}{1 + e^{-x}}$$

- (b) Filter the F_B activations through W to F_C using the (**sigmoid**) equation

$$c_j = f\left(\left(\sum_{i=1}^p b_i w_{ij}\right) + \Gamma_j\right)$$

for all $j = 1, 2, \dots, q$, where c_j is the activation value of the j th F_C PE, Γ_j is the j th F_C PE's threshold value.

- (c) Compute the discrepancy (error) between the computed and desired F_C PE values using the equation

$$d_j = c_j(1 - c_j)(c_j^k - c_j)$$

for all $j = 1, 2, \dots, q$, where d_j is the j th F_C PE's computed error.

- (d) Calculate the error of each F_B PE relative to each d_j with the equation for all $i = 1, 2, \dots, p$, where e_i is the i th F_B PE's computed error.

$$e_i = b_i(1 - b_i) \sum_{j=1}^q w_{ij} d_j$$

- (e) Adjust the F_B to F_C connections for all $i = 1, 2, \dots, p$, where $j = 1, 2, \dots, q$. α is a positive constant controlling the learning rate.

$$w_{ij}^{new} = w_{ij}^{old} + \Delta w_{ij}$$

$$\Delta w_{ij} = \alpha b_i d_j$$

- (f) Adjust the F_C thresholds for all $j = 1, 2, \dots, q$, where $\Delta \Gamma_j$ is the amount of change to the j th F_C PE's threshold value.

$$\Gamma_j^{new} = \Gamma_j^{old} + \Delta \Gamma_j \quad \text{where} \quad \Delta \Gamma_j = \alpha d_j$$

(g) Adjust the F_A to F_B connections for all $h = 1, 2, \dots, n$, and all $i = 1, 2, \dots, p$.

β is a positive constant controlling the learning rate.

$$v_{hi}^{new} = v_{hi}^{old} + \Delta v_{hi} \quad \text{where } \Delta v_{hi} = \beta a_h e_i$$

(h) Adjust the F_B thresholds by

$$\theta_i^{new} = \theta_i^{old} + \Delta \theta_i \quad \text{where } \Delta \theta_i = \beta e_i$$

for all $i = 1, 2, \dots, n$, where $\Delta \theta_i$ is the amount of change to the i th F_C PE's threshold value.

3. Repeat step (2) until the error correction value d_j , for each $j = 1, 2, \dots, q$, and each $k = 1, 2, \dots, m$, is either sufficiently low or zero.

4. One forward run of the ANN (also called Backpropagation recall) accepts an input at the F_A PEs and feeds it through the V and W matrices to produce a response at the F_C PEs. This feedforward operation creates F_B PEs (activations) using

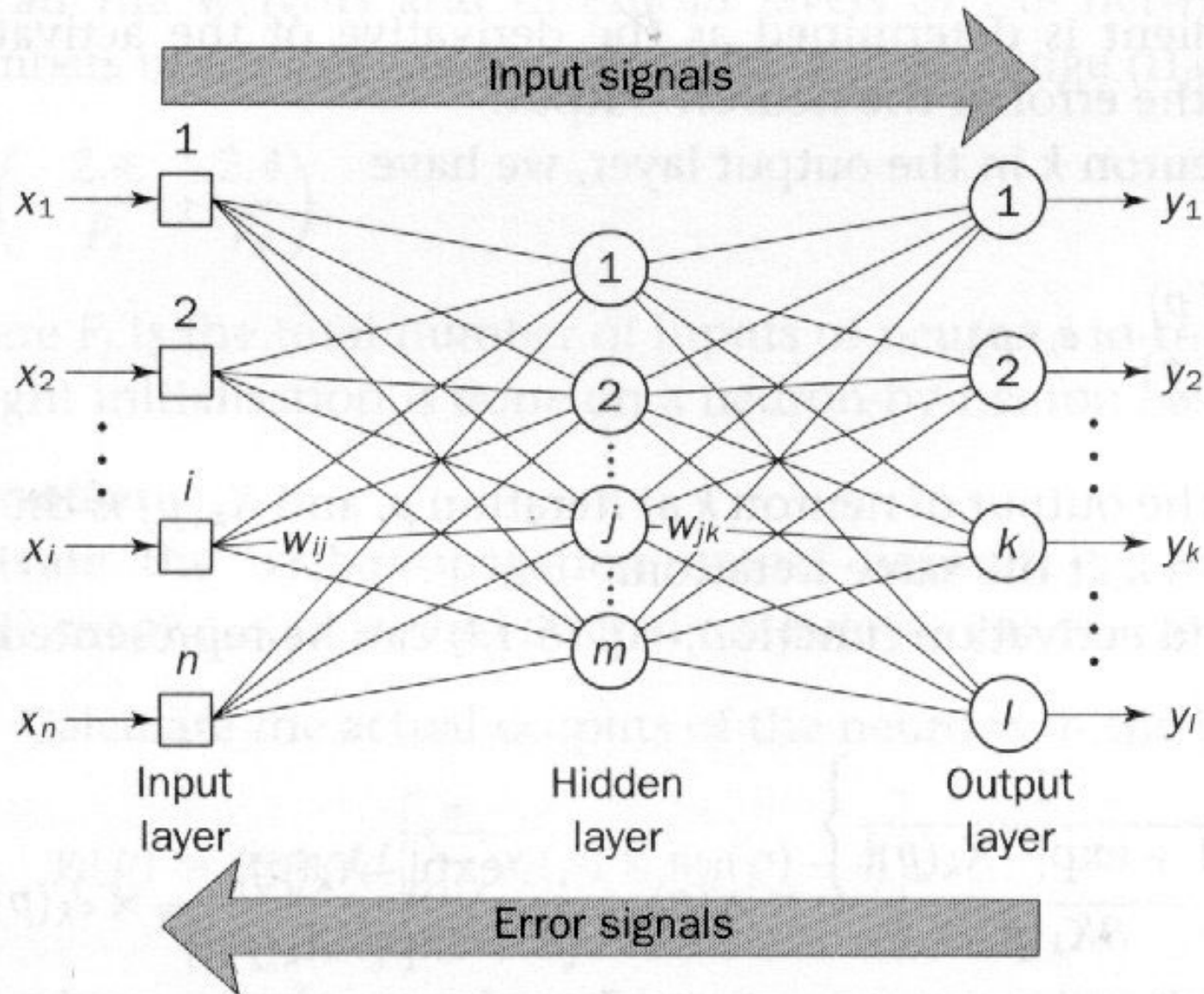
$$b_i = f\left(\left(\sum_{h=1}^n a_h v_{hi}\right) + \theta_i\right)$$

for $i = 1, 2, \dots, p$. Once the F_B PE values have been all calculated they are then used to create new F_C PE values using

$$c_j = f\left(\left(\sum_{i=1}^p b_i w_{ij}\right) + \Gamma_j\right)$$

for all $j = 1, 2, \dots, q$.

Why is it called BP ?

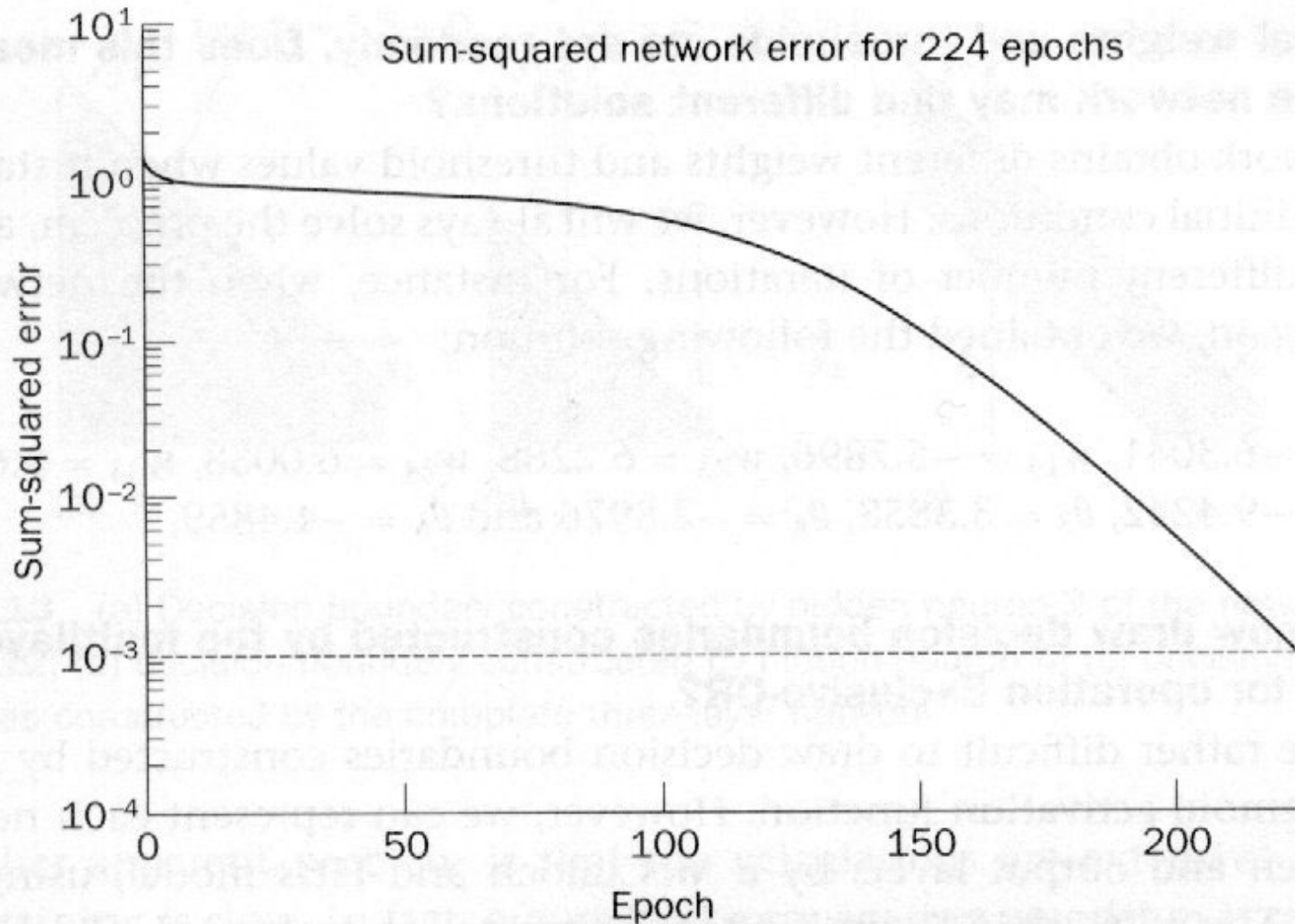


Why do we need to sum the squared error ?

$$E = \frac{1}{2} \sum_{j=1}^q (c_j^k - c_j)^2$$

- It is a useful indicator of the network's performance. The BP training algorithm attempts to minimize this criterion.
- When this value is an entire pass through all training sets, or **epoch**, is sufficiently small, a network is considered to have converged.

Learning curve: the sum-squared error plotted versus the number of epochs



Example of BP

- In the next **Example**, twenty-one data points of an unknown function ($y=f(x)$) is given. The aim is to design an ANN (**logsig-purelin**) to learn the function from the data points.
- Five hidden units is attempted first, with both learning rates set to 0.3.
- Figure 1 shows the convergence curve.
- Figure 2 shows the result from the trained ANN.

Regression Problem: Function Approximation

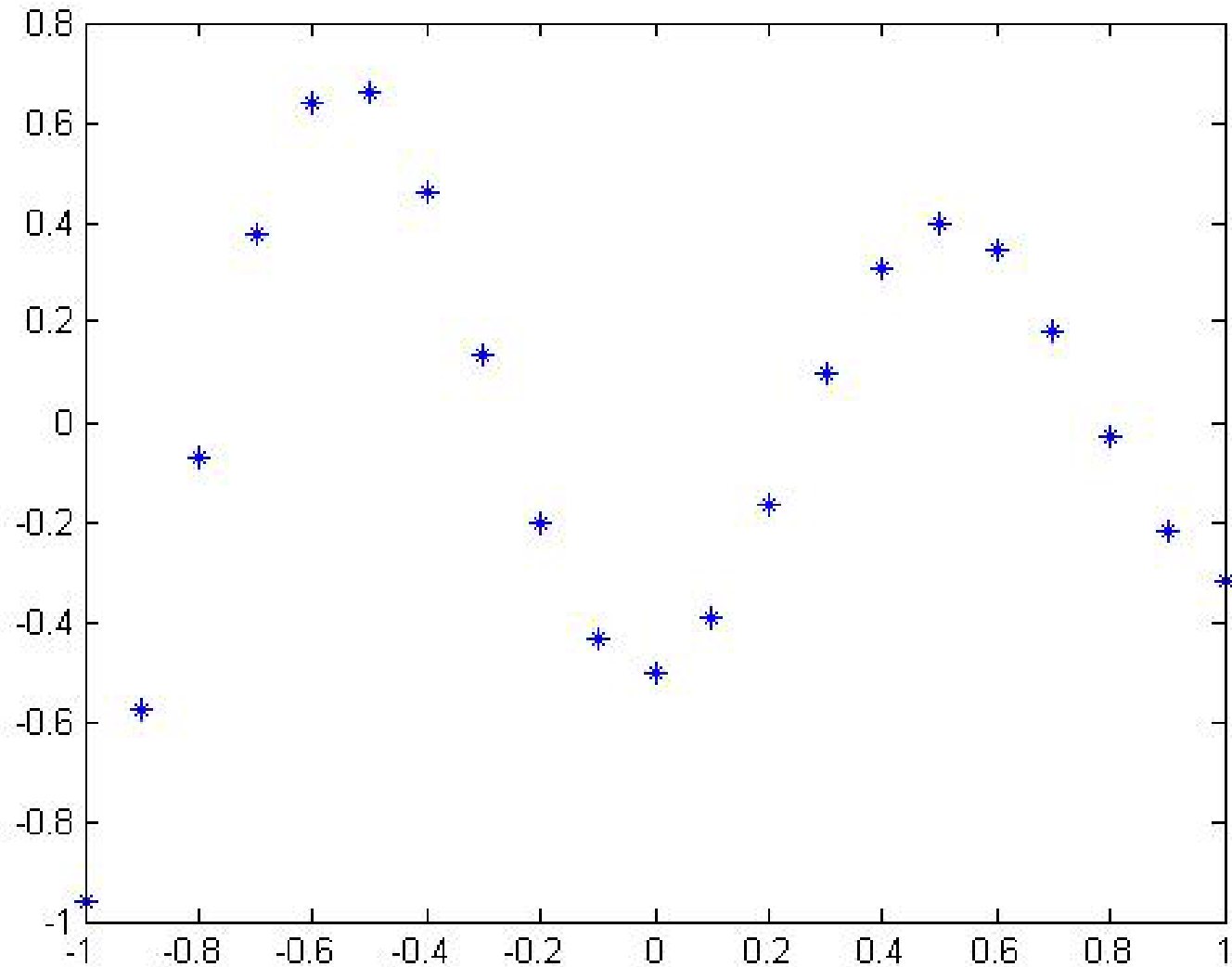


Figure 1

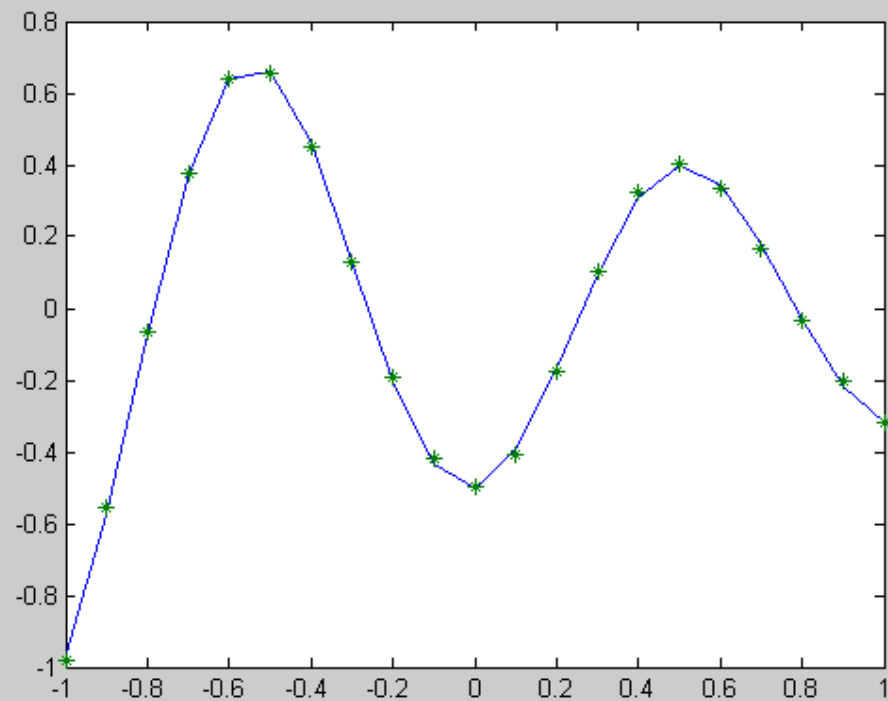
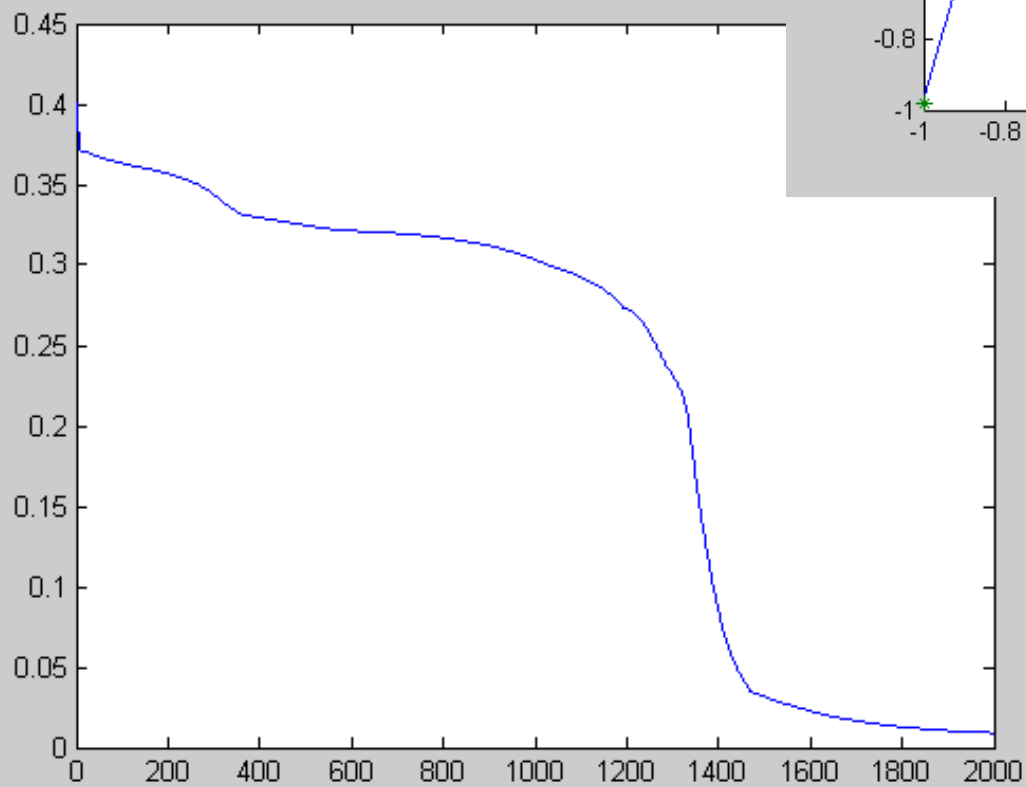
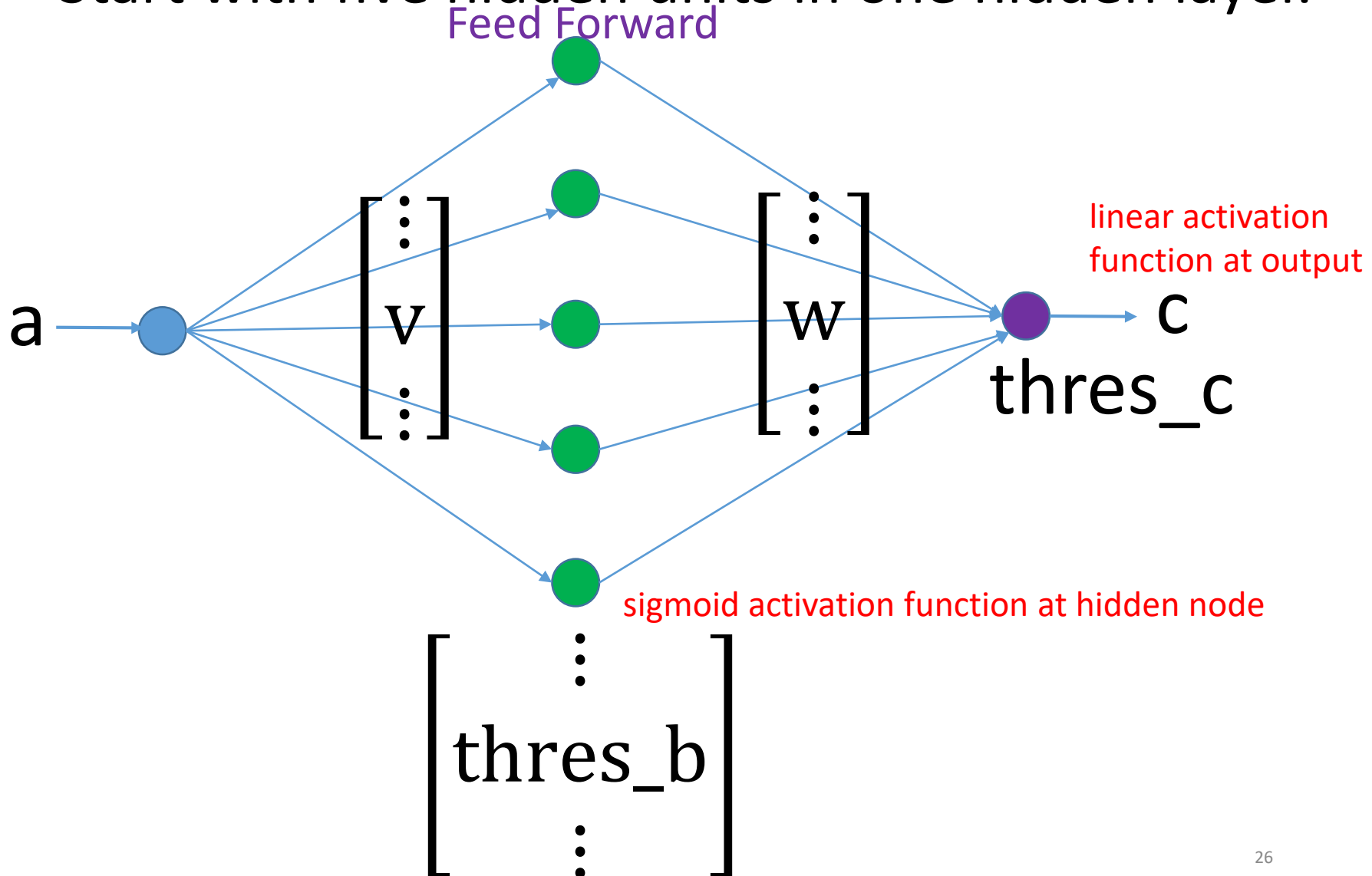


Figure 2

Regression Problem of one-input, one output
 Start with five hidden units in one hidden layer.



Hidden layer outputs (vector form)

$b = \text{sigmoid}(\text{np.dot}(a, v) + \text{thres_b})$

Output layer outputs (vector form)

$c = \text{np.dot}(b, w) + \text{thres_c}$

Feed Forward Equations

Hidden layer outputs

$b1 = \text{sigmoid}(a*v1 + \text{thres_b1})$

$b2 = \text{sigmoid}(a*v2 + \text{thres_b2})$

$b3 = \text{sigmoid}(a*v3 + \text{thres_b3})$

$b4 = \text{sigmoid}(a*v4 + \text{thres_b4})$

$b5 = \text{sigmoid}(a*v5 + \text{thres_b5})$

Output layer output

$c = b1*w1 + b2*w2 + b3*w3 + b4*w4 + b5*w5 + \text{thres_c}$

Python-Implementation of the BP algorithm with sigmoid hidden units and purelin output units

```
# Python implementation of Artificial Neural Network (ANN)  
# Dr. G. Pang, August 2025
```

```
import numpy as np  
import copy
```

ANN_Python_Code.ipynb

```
def sigmoid(x):  
    """Sigmoid activation function."""  
    return 1.0 / (1.0 + np.exp(-x))
```

```
def bp_logsig_lin(inputs, outputs, num_hid, alpha, beta, err, max_stop):  
    """  
    Backpropagation Algorithm for an ANN with one hidden layer  
    LOGSIG function is used in hidden layer;  
    LINEAR function is used in output layer.  
  
    Args:  
        inputs (numpy.ndarray): Input matrix of shape (samples, n).  
        outputs (numpy.ndarray): Output matrix of shape (samples, q).  
        num_hid (int): Number of hidden units (i.e., p).  
        alpha (float): Learning rate between the output and hidden layer.  
        beta (float): Learning rate between the input and hidden layer.  
        err (float): Error for the stopping criterion.  
        max_stop (int): Maximum number of iterations.  
  
    Returns:  
        tuple: (convergence, ave_err, max_err, result, v, w, thres_b, thres_c)  
    """  
    print("Demo on Backpropagation Algorithm by Dr. G. Pang")
```

Defining
an ANN
function
in python

```

# Dimensions of input and output matrices
num_examples, num_in = inputs.shape
_, num_out = outputs.shape

print('number of training samples = ', num_examples)
print('number of input = ', num_in)
print('number of output = ', num_out)

if num_examples != outputs.shape[0]:
    raise ValueError("Number of input-output training examples not the same")

# Initialize convergence vector
convergence = np.zeros(max_stop)

# Initialize weights and thresholds randomly
v = np.random.rand(num_in, num_hid) - 0.5
w = np.random.rand(num_hid, num_out) - 0.5
thres_b = np.random.rand(1, num_hid) - 0.5
thres_c = np.random.rand(1, num_out) - 0.5

# Initialize variables
iterations = 0
max_err = 1000

```

Error checking !

Weights and Biases
Initialization

```

# Main training loop
while max_err >= err and iterations < max_stop:
    iterations += 1

    for n in range(num_examples):
        # Forward pass
        a = inputs[n, :]
        ck = outputs[n, :]

        # Hidden layer outputs
        b = sigmoid(np.dot(a, v) + thres_b)

        # Output layer outputs
        c = np.dot(b, w) + thres_c

        # Compute errors at the output
        d = ck - c

        # Compute errors at the hidden layer
        e = b * (1 - b) * np.dot(d, w.T)

        # Update weights and thresholds
        w += alpha * np.outer(b, d)
        thres_c += alpha * d

        v += beta * np.outer(a, e)
        thres_b += beta * e

```

Get the inputs and targeted output

“a” to “b”

“b” to “c”

Calculate d, the error “diff”

Calculate e, another error term

Update w and thres_c

gives a 2D matrix, added to 2D matrix w

Update v and thres_d

```
# Recall phase
```

```
save_err = np.zeros((num_examples, num_out))
```

```
for n in range(num_examples):
```

```
    a = inputs[n, :]
```

```
    ck = outputs[n, :]
```

```
    # Hidden layer outputs
```

```
    b = sigmoid(np.dot(a, v) + thres_b)
```

```
    # Output layer outputs
```

```
    c = np.dot(b, w) + thres_c
```

```
    save_err[n, :] = np.abs(ck - c)
```

Calculation at hidden layer

After one iteration:
Feedback Forward
to obtain error of
each case. Get max
and average error.

Calculation at output

```
# Calculate average and maximum errors
```

```
ave_err = np.sum(save_err) / (num_examples * num_out)
```

```
max_err = np.max(save_err)
```

```
print(f"Iteration: {iterations}, Average Error: {ave_err},  
      Max Error: {max_err}")
```

```
convergence[iterations - 1] = ave_err
```

Just finished one iteration

```

print("Stopping criterion has been reached!")
print("The results after training: inputs, desired outputs, NN outputs")

# Final recall
result = np.zeros((num_examples, num_out))
for n in range(num_examples):
    a = inputs[n, :]
    b = sigmoid(np.dot(a, v) + thres_b)
    result[n, :] = np.dot(b, w) + thres_c

return convergence, ave_err, max_err, result, v, w, thres_b, thres_c

```

After all iterations:
Feedback Forward
to obtain max and
average error.

```

P = np.linspace(-1,1,21)
print(P.shape)

```

AN EXAMPLE

```

T=np.array([ -0.96,  -0.577,  -0.0729,  0.377,  0.641,
             0.66 ,  0.461 ,  0.1336 , -0.201,  -0.434,
            -0.5 , -0.393,  -0.1647,  0.0988,  0.3072,
             0.396 ,0.3449 ,  0.1816,  -0.0312,  -0.2189, -0.3201])

```

(21,)

```
# parameters
```

```
num_hid = 5
```

```
alpha = 0.3
```

```
beta = 0.3
```

```
err = 0.005
```

```
max_stop = 2000
```

```
inputs = copy.deepcopy(P)
```

input

```
outputs = copy.deepcopy(T)
```

output

```
inputs = inputs.reshape(21,1)
```

```
outputs = outputs.reshape(21,1)
```

← The hyper-parameters

The ANN function

```
convergence, ave_err, max_err, result, v, w, thres_b, thres_c = bp_logsig_lin(  
    inputs, outputs, num_hid, alpha, beta, err, max_stop)
```

Demo on Backpropagation Algorithm by Dr. G. Pang

number of training samples = 21

number of input = 1

number of output = 1

Iteration: 1, Average Error: 0.3994288464436024, Max Error: 0.887004049747115

Iteration: 2, Average Error: 0.3970350484189459, Max Error: 0.876813698727579

```
import matplotlib.pyplot as plt
```

```
show_result = result.reshape(21,1)
```

```
plt.figure(figsize=(8,10))
```

```
plt.plot(inputs,T,label = 'training data',marker='o',color = 'blue')
```

```
plt.scatter(inputs,result,label = 'result',marker='*',color = 'red')
```

```
plt.title('Plotting of the training data points and result')
```

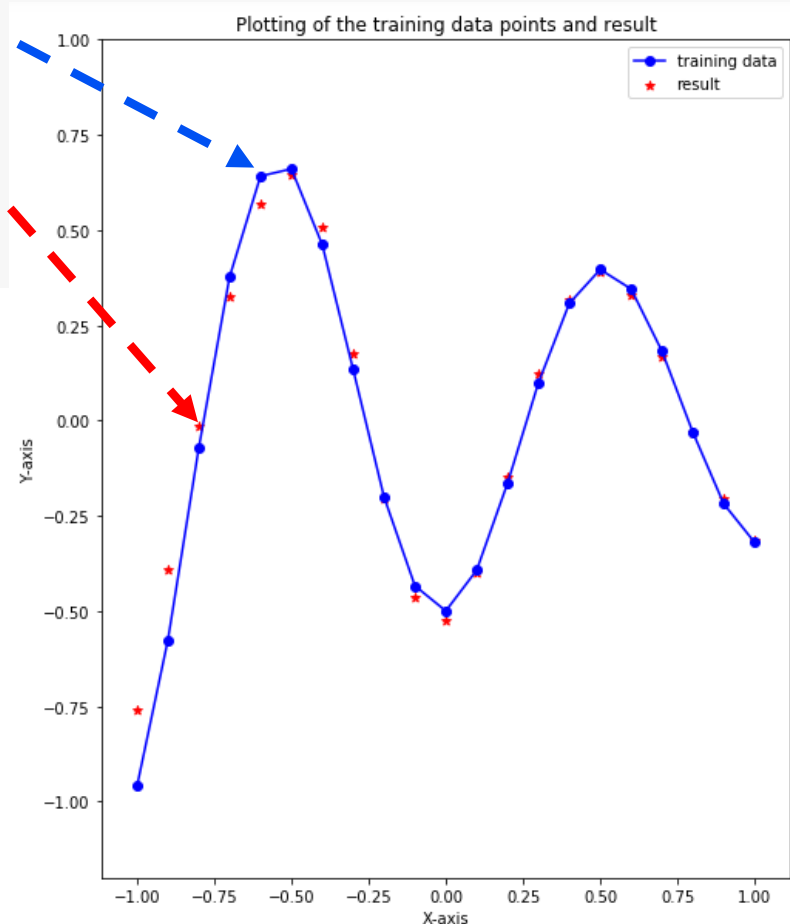
```
plt.xlabel('X-axis')
```

```
plt.ylabel('Y-axis')
```

```
plt.ylim(-1.2,1)
```

```
plt.legend()
```

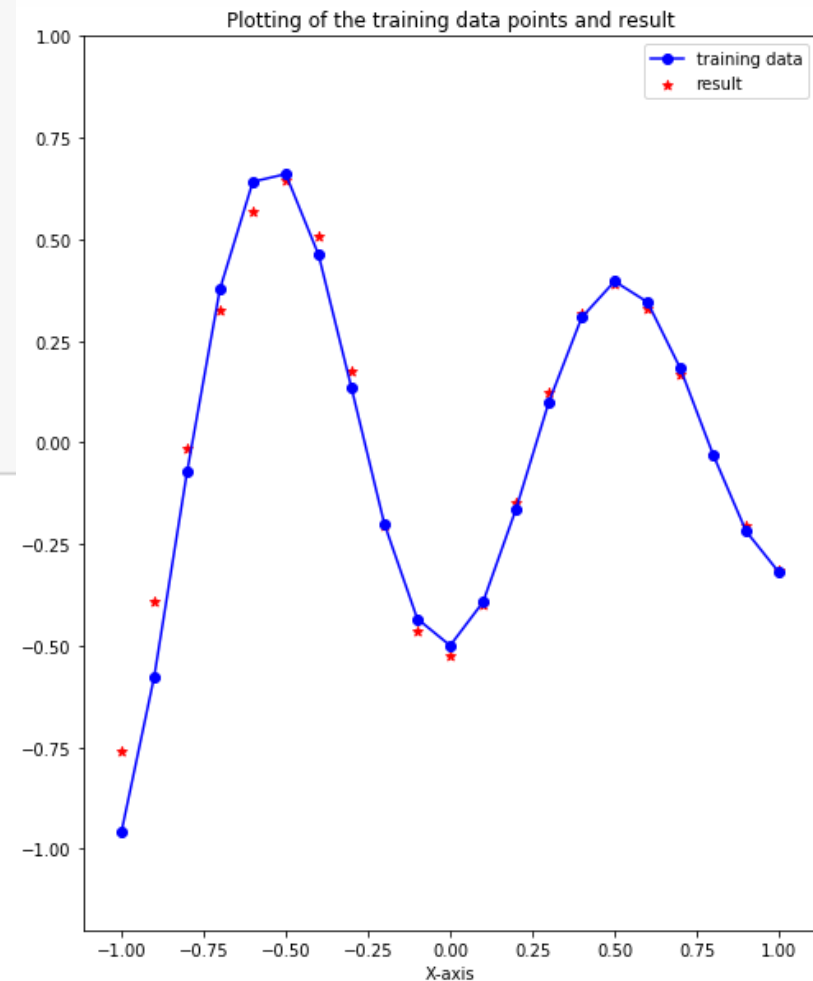
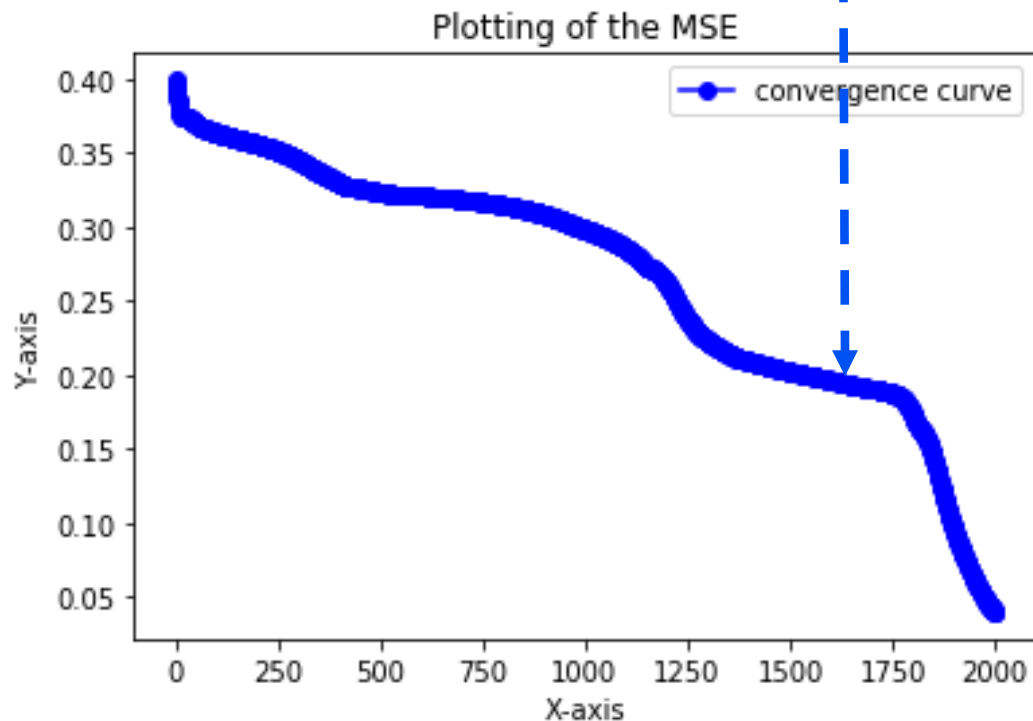
```
plt.show()
```



```
# To show the convergence curve, or  
# the variation of MSE during iterations
```

```
plt.plot(np.arange(0, len(convergence)),  
         convergence, label = 'convergence curve',  
         marker='o', color = 'blue')
```

```
plt.title('Plotting of the MSE')  
plt.xlabel('X-axis')  
plt.ylabel('Y-axis')  
plt.legend()  
plt.show()
```



```
# What are the "results" from training ?
```

```
#
```

```
print ('v = ', v)
```

```
print ('w = ', w)
```

```
print ('thres_b = ', thres_b)
```

```
print ('thres_c = ', thres_c)
```

```
v = [[6.83086836 7.37993909 3.75844048 2.46369124 7.64386298]]
```

```
w = [[-1.62825522]
```

```
 [ 1.65276462]
```

```
 [ 4.37760262]
```

```
 [ 0.94035243]
```

```
 [-3.17025827]]
```

The “results” from training

```
thres_b = [[-5.04059715 -1.55430492  3.03021211 -2.72894328  2.21488648]]
```

```
thres_c = [[-2.17689899]]
```

Conclusions: BP algorithm for training of multilayer feedforward ANN

- Numerous researchers, under different constraints, have shown that multilayer feedforward **ANNs are capable of approximating any finite function to any degree of accuracy.**
- Networks with a single hidden layer are capable of approximating the class of "useful" functions.
- ANN is also powerful for general classification task.
- Very general with many variations (model parameters, hyper-parameters) and many applications
- *Conclusion:* *Multilayer feedforward structure leads to the general function approximation and classification tasks. There are many further structures and variations.*

What is the regression issue ?

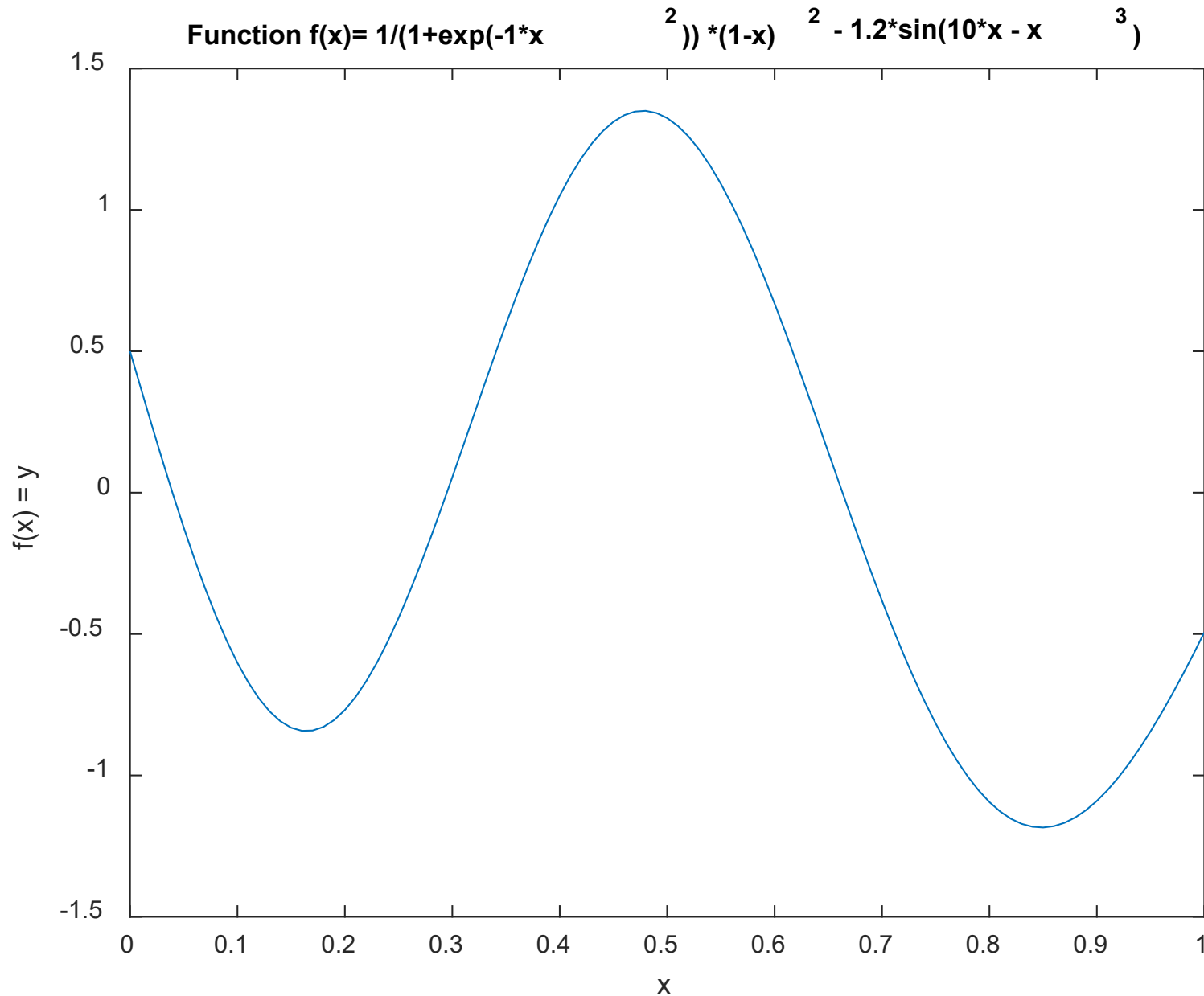
Given the **input-output pattern**, find the network that relates the input and output of the **unknown** system.

x1	x2	x3		y1	y2
0.9501	0.8214	0.9355		0.1389	0.4451
0.2311	0.4447	0.9169		0.2028	0.9318
0.6068	0.6154	0.4103		0.1987	0.4660
0.4860	0.7919	0.8936		0.6038	0.4186
0.8913	0.9218	0.0579		0.2722	0.8462
0.7621	0.7382	0.3529		0.1988	0.5252
0.4565	0.1763	0.8132		0.0153	0.2026
0.0185	0.4057	0.0099		0.7468	0.6721



0.8381	0.0196	0.6813		???	???
--------	--------	--------	--	-----	-----

Optional Exercise: Regression Problem



Optional Exercise: Regression Problem

Exercise:

- Use a number of training data to obtain an ANN that learns the function.
- Change the number of hidden units and run the code again to see the difference.
- You may also want to vary the number of training data.

